How Many Tries Does It Take? Iterative Self-Repair in LLM Code Generation Across Model Scales and Benchmarks

Johin Johny Arimbur
Independent Researcher
johinjohny144@gmail.com

***Abstract*—Large language models frequently fail to produce correct code on their first attempt, yet most benchmarks evaluate them in a single-shot setting. We investigate *iterative self-repair*—feeding execution errors back to the model for correction—across seven models spanning three families and both open-weight and proprietary providers: Llama 3.1 8B, Llama 3.3 70B, Llama 4 Scout (MoE, 16 experts), Llama 4 Maverick (MoE, 128 experts), Qwen3 32B, Gemini 2.5 Flash, and Gemini 2.5 Pro. On HumanEval (164 problems) and MBPP Sanitized (257 problems) with up to five attempts, self-repair universally improves pass rates: +4.9 to +17.1 pp on HumanEval and +16.0 to +30.0 pp on MBPP. Gemini 2.5 Flash achieves the highest final pass rates (96.3% HumanEval, 93.8% MBPP). Most gains concentrate in the first two rounds. Error-type analysis shows assertion errors (logical mistakes) are the hardest to repair at ∼45%, while syntax and name errors are repaired at substantially higher rates, connecting to broader findings on the limits of LLM self-correction. To our knowledge, this is the first systematic comparison of self-repair across dense and MoE architectures spanning three model families (Meta, Alibaba, Google), and the first token-efficiency comparison against independent resampling. A prompt ablation reveals chain-of-thought repair yields up to +5.5 pp additional repair gain over minimal prompting for capable models.**

***Index Terms*—large language models, code generation, self-repair, iterative refinement, HumanEval, MBPP, self-debugging**

I. Introduction

The ability of large language models (LLMs) to generate functionally correct code from natural-language specifications has improved dramatically in recent years [1]–[3]. Benchmarks such as HumanEval [1] and MBPP [4] have become standard yardsticks, typically reporting a *pass@1* metric: the fraction of problems solved correctly on a single attempt. While informative, this single-shot evaluation protocol diverges sharply from how human programmers actually write code. A professional developer routinely writes an initial draft, runs it, reads the error message, and iterates—often multiple times—before arriving at a correct solution.

Iterative self-repair replicates this workflow within an LLM pipeline. When a generated program fails its test suite, the model receives the error traceback and is asked to produce a corrected version. This cycle can repeat for a fixed number of rounds. Prior work has explored self-repair with proprietary models such as GPT-3.5 and GPT-4 [5], [6], yielding mixed conclusions about its effectiveness. Olausson et al. [5] notably argued that self-repair is "not a silver bullet," observing that weaker models may introduce new errors during repair attempts.

However, the landscape has changed significantly since those studies. A new generation of open-weight models (2024–2025) exhibits substantially stronger instruction-following and error-comprehension capabilities, and novel architectures such as mixture-of-experts (MoE) have become prevalent. No prior work has evaluated self-repair on these modern models or compared dense vs. MoE architectures for repair. These developments motivate a fresh empirical investigation: *does self-repair now work universally, even for smaller models? How do dense and MoE architectures differ in their repair dynamics? And what practical guidelines can we derive for deployment?*

In this paper, we address these questions with a systematic evaluation across seven models from three families: Llama 3.1 8B (8B dense) [7], Llama 3.3 70B (70B dense) [8], Llama 4 Scout 17B (17B active, MoE with 16 experts) [9], Llama 4 Maverick 17B (17B active, MoE with 128 experts) [9], Qwen3 32B (32B dense) [10], Gemini 2.5 Flash [11], and Gemini 2.5 Pro [11]. The five open-weight models are accessed via the Groq free-tier API; the two Gemini models are accessed via Google Cloud Vertex AI. All use greedy decoding (temperature $= 0.0$) for reproducibility. We evaluate on two benchmarks—HumanEval (164 problems) and MBPP Sanitized (257 problems)—with up to five attempts per problem.

Our contributions are as follows:

1) We demonstrate that self-repair is now *universally* effective across seven models from three families—even at 8B scale—updating the finding by Olausson et al. [5] that weaker models are harmed by self-repair. This is, to our knowledge, the first systematic study of self-repair spanning open-weight and proprietary models across three families (Meta, Alibaba, Google), including MoE architectures.
2) We present the first direct comparison of dense and mixture-of-experts architectures for self-repair across multiple model families, revealing distinct interaction patterns: on HumanEval, Scout (16 experts) achieves the highest repair gain among open-weight models (+14.0 pp), while the Gemini models establish new ceiling performance levels.

3) We provide cross-benchmark validation on both HumanEval and MBPP Sanitized, showing consistent model rankings and diminishing-returns patterns, strengthening generalizability beyond single-benchmark studies.
4) We present fine-grained error-type analysis revealing that error category is a strong predictor of repair success: name errors are repaired at ∼77% rates, while assertion errors succeed at only ∼45%, connecting to the broader literature on LLM self-correction limits [12].
5) We quantify the diminishing-returns curve and derive practical deployment guidelines: two repair rounds capture the majority (76–94%) of achievable gains, and error-type-based routing can optimize the repair-vs-resample tradeoff.
6) We conduct a repair prompt ablation comparing minimal, explain-then-fix, and chain-of-thought strategies, finding that CoT prompting yields +5.5 pp additional repair gain over minimal prompting for the 70B model.

II. Related Work

A. Code Generation with LLMs

The emergence of Codex [1] demonstrated that LLMs fine-tuned on code repositories could generate functionally correct programs from docstrings. Subsequent work scaled this capability through larger models [2], code-specific pre-training [3], and reinforcement learning from execution feedback [13]. The HumanEval benchmark [1] and the Mostly Basic Programming Problems (MBPP) dataset [4] have become standard evaluation suites, while more challenging benchmarks such as APPS [14] and SPoC [15] test competition-level reasoning.

B. Self-Debugging and Self-Repair

Chen et al. [6] proposed *Self-Debug*, demonstrating that LLMs can identify and fix bugs in their own code when provided with execution results. Their approach included several feedback strategies: simple error messages, code explanations, and unit test traces. Olausson et al. [5] conducted a thorough investigation of self-repair, concluding that while GPT-4 benefited substantially, weaker models like GPT-3.5 often failed to improve or even degraded in performance. Our work extends this line of inquiry to seven models from three families across two benchmarks, revealing that the "weaker models fail" conclusion no longer holds for the 2024–2025 model generation.

C. Iterative Refinement Beyond Code

The principle of iterative self-improvement has been explored in broader contexts. Madaan et al. [16] introduced *Self-Refine*, a general framework in which LLMs iteratively critique and revise their own outputs across diverse tasks. Shinn et al. [17] proposed *Reflexion*, using verbal reinforcement signals to improve agent performance over multiple episodes. However, Huang et al. [12] cautioned that LLMs cannot reliably self-correct *reasoning* without external feedback, an observation consistent with our finding that assertion errors (which require reasoning corrections) are the hardest to repair.

D. Multi-Turn and Collaborative Code Generation

Recent work has also investigated multi-turn interactions for code generation. Key et al. [18] explored multi-turn code generation with instruction-tuned models, while Zhang et al. [19] investigated self-collaboration patterns with ChatGPT. Zhong et al. [20] proposed LDB, a debugger that verifies runtime execution step-by-step. SelfEvolve [21] used LLMs to iteratively evolve code through self-generated feedback. Our work differs in its focus on the simplest possible self-repair protocol (feeding back raw error messages) and its systematic comparison across seven models from three families and two benchmarks.

III. Methodology

A. Self-Repair Protocol

Our iterative self-repair protocol operates as follows. Given a programming problem specified by a function signature and docstring (as in HumanEval and MBPP), the model generates an initial code solution at round R_0. This solution is extracted from the model's response, combined with the corresponding test cases, and executed in a sandboxed Python environment. If all test cases pass, the problem is marked as solved. If execution produces an error, we capture the error type and traceback message, then construct a repair prompt containing: (1) the original problem specification, (2) the model's previous code attempt, and (3) the error message. The model is then asked to generate a corrected solution. This cycle repeats for up to four repair rounds (R_1 through R_4), for a maximum of five total attempts per problem.

Formally, for a problem p and model M, let $c_0 = M(p)$ be the initial code generation. If $\texttt{exec}(c_0)$ succeeds, we record success at R_0. Otherwise, let $e_0 = \texttt{error}(c_0)$ be the captured error. For each subsequent round $i \in \{1, 2, 3, 4\}$:

$$c_i = M(p, c_{i-1}, e_{i-1}) \tag{1}$$

where the model receives the problem, its previous attempt, and the error message. If $\texttt{exec}(c_i)$ succeeds at any round, the problem is marked as solved at round R_i and no further attempts are made.

B. Code Extraction

LLM outputs frequently contain natural-language explanations, markdown formatting, and other non-code content surrounding the actual solution. We employ a robust code extraction pipeline that: (1) searches for Python code blocks delimited by markdown fences (```python ... ```); (2) falls back to extracting the longest contiguous block of syntactically valid Python if no fences are found; and (3) removes any extraneous content such as chain-of-thought traces (e.g., <think>...</think> tags) that some reasoning models may prepend to their outputs.

C. Error Classification

When a solution fails, we classify the error into one of the following categories based on the Python exception type:

- **AssertionError**: The code runs without crashing but produces incorrect output, failing one or more test assertions. These errors indicate logical mistakes.
- **SyntaxError**: The generated code is not valid Python.
- **TypeError / ValueError**: Runtime type mismatches or invalid values, often indicating misunderstanding of the expected input/output types.
- **NameError**: References to undefined variables or functions, typically caused by incomplete code generation.
- **IndexError / KeyError**: Out-of-bounds access or missing dictionary keys, indicating edge-case handling failures.
- **Timeout**: The code does not terminate within the allotted time, suggesting infinite loops or excessive computational complexity.

D. Metrics

Our primary metric is *cumulative pass@1* at each round R_i, defined as the fraction of problems solved by round R_i (inclusive of all preceding rounds). Because each problem receives exactly one attempt per round with greedy decoding (temperature $= 0.0$), pass@1 is deterministic and does not require the unbiased estimator used for stochastic sampling [1]. We also report the *self-repair gain* Δ, defined as the difference between the final cumulative pass@1 (at R_4) and the initial pass@1 (at R_0), measured in percentage points (pp). We note that on HumanEval ($n = 164$), a single problem corresponds to $\approx$0.6 pp; differences smaller than $\sim$2 pp (3 problems) should be interpreted cautiously.

IV. Experimental Setup

A. Models

We evaluate seven models from three families, spanning open-weight and proprietary providers:

1) Open-Weight Models (Groq API):

1) **Llama 3.1 8B** [7]: An 8-billion-parameter dense transformer from Meta. This serves as our smallest baseline and represents the class of efficient, instruction-tuned models commonly deployed in resource-constrained settings.
2) **Llama 3.3 70B** [8]: A 70-billion-parameter dense transformer from Meta. This is the largest dense open-weight model in our evaluation and provides a strong baseline for comparing against mixture-of-experts architectures.
3) **Llama 4 Scout 17B** [9]: A mixture-of-experts (MoE) model from Meta with 17 billion active parameters distributed across 16 experts. The MoE architecture allows the model to maintain a large total parameter count while activating only a fraction of parameters per token.
4) **Llama 4 Maverick 17B** [9]: A mixture-of-experts model from Meta with 17 billion active parameters and 128 experts. Compared to Scout, Maverick uses significantly more experts, providing an interesting comparison point for how MoE granularity affects code generation and self-repair.
5) **Qwen3 32B** [10]: A 32-billion-parameter dense transformer from Alibaba. Qwen3 supports both reasoning (thinking) and non-reasoning modes. Thinking mode was disabled via the `/no_think` setting to ensure fair comparison with non-reasoning models.

2) Proprietary Models (Vertex AI):

6) **Gemini 2.5 Flash** [11]: A lightweight model from Google optimized for speed and cost-efficiency. Gemini 2.5 Flash is designed as a fast-inference model suitable for high-throughput applications, making it an interesting comparison for self-repair efficiency.
7) **Gemini 2.5 Pro** [11]: Google's flagship reasoning model with extended "thinking" capabilities. Unlike Qwen3's optional thinking mode, Gemini 2.5 Pro integrates reasoning natively. This model represents the frontier of proprietary model capability in our evaluation.

The inclusion of Gemini models extends our evaluation from two model families (Meta, Alibaba) to three (Meta, Alibaba, Google), and from open-weight-only to a mix of open-weight and proprietary models. This broadens the generalizability of our findings.

All models are queried with greedy decoding (temperature $=$ 0.0) to ensure deterministic, reproducible outputs. A minimal system prompt is used for all models: *"You are an expert Python programmer. Complete the given function. Return ONLY the Python code, no explanations, no markdown formatting."* No few-shot examples are provided. For initial generation, each model receives only the function signature and docstring. For repair rounds, the model receives the original problem, its previous code attempt, and the error message. The repair prompt is minimal: it presents the error and asks for a corrected function (see Section III).

B. Benchmarks

We evaluate on two standard code generation benchmarks:

1) **HumanEval** [1]: 164 hand-written Python programming problems, each with a function signature, docstring, and test cases. Problems range from simple string manipulations to moderately complex algorithmic tasks. All 164 problems were evaluated for all seven models.
2) **MBPP Sanitized** [4]: 257 problems from the Mostly Basic Programming Problems dataset (sanitized subset). MBPP problems tend to be shorter and more focused than HumanEval, but cover a broader range of programming concepts. All 257 problems were evaluated for all seven models.

We acknowledge that these benchmarks are considered relatively easy by 2025 standards. We discuss the implications of this choice and the need for evaluation on harder benchmarks in Section VII.

TABLE I
CUMULATIVE PASS@1 (%) ON HUMANEVAL (164 PROBLEMS) BY REPAIR ROUND. R_0 IS THE INITIAL ATTEMPT; R_1–R_4 ARE REPAIR ROUNDS. Δ DENOTES THE TOTAL SELF-REPAIR GAIN ($R_4 - R_0$) IN PERCENTAGE POINTS.

Model	Family	R_0	R_1	R_2	R_3	R_4	Δ
Llama 3.1 8B	Meta	67.1	73.2	75.6	76.2	76.8	+9.8
Llama 3.3 70B	Meta	82.9	89.6	90.9	92.7	93.3	+10.4
Scout 17B (16E)	Meta	75.6	84.8	87.2	89.0	89.6	+14.0
Maverick 17B (128E)	Meta	87.2	91.5	92.7	93.9	93.9	+6.7
Qwen3 32B	Alibaba	87.8	90.2	92.1	92.7	92.7	+4.9
Gemini 2.5 Flash	Google	86.6	95.7	95.7	96.3	96.3	+9.8
Gemini 2.5 Pro	Google	73.2	82.9	86.6	89.6	90.2	+17.1

C. Infrastructure

Open-weight model experiments were conducted using the Groq free-tier API, which provides rate-limited access to hosted models at no monetary cost. Gemini experiments were conducted using Google Cloud Vertex AI, with total Gemini API costs under $5 USD. Code execution was performed in isolated Python subprocesses with a 15-second timeout per test case. All code and results are publicly available.[1]

V. RESULTS

A. HumanEval Results

Table I presents the cumulative pass@1 rates on HumanEval across all repair rounds for all seven models. Several key observations emerge:

Universal improvement. Self-repair improves pass rates for every model tested, across all three families and both open-weight and proprietary models. Every model gains at least +4.9 pp.

Gemini 2.5 Flash achieves the highest final rate. At 96.3%, Flash surpasses all open-weight models, including Maverick (93.9%) and Llama 3.3 70B (93.3%). Its R1 jump from 86.6% to 95.7% (+9.1 pp in a single round) is among the largest first-round gains (comparable to Pro's +9.7 pp and Scout's +9.2 pp), suggesting strong error comprehension.

Gemini 2.5 Pro shows the largest repair gain. Pro achieves +17.1 pp, the highest Δ on HumanEval. However, its lower initial pass rate (73.2%) reflects overhead from its reasoning traces, which occasionally interfere with code extraction. Its final rate (90.2%) is competitive but below Flash, illustrating that stronger reasoning does not always translate to better code generation in a simple prompt-completion paradigm.

Diminishing returns from higher baselines. Among the open-weight models, the pattern holds: Qwen3 32B starts at 87.8% and gains only +4.9 pp, while Scout starts lower at 75.6% and gains +14.0 pp. Flash is a notable exception—it starts high (86.6%) yet still achieves a substantial +9.8 pp gain, suggesting that its repair capability exceeds the diminishing-returns trend.

Figure 1 visualizes these trajectories. The curves illustrate the universal upward trend, with Flash breaking through the ∼94% ceiling observed among the open-weight models.

[1] https://github.com/Johin2/Self-repair

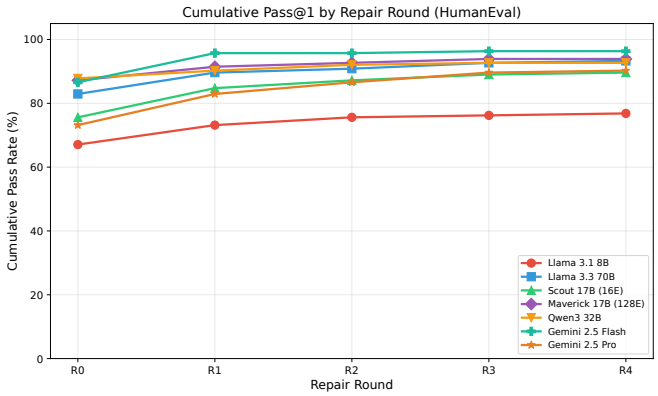

Fig. 1. Cumulative pass@1 rate on HumanEval across repair rounds for all seven models. Gemini 2.5 Flash achieves the highest final rate (96.3%), while most gains concentrate in the first two rounds.

TABLE II
CUMULATIVE PASS@1 (%) ON MBPP SANITIZED (257 PROBLEMS) BY REPAIR ROUND.

Model	Family	R_0	R_1	R_2	R_3	R_4	Δ
Llama 3.1 8B	Meta	55.6	66.9	69.6	70.8	71.6	+16.0
Llama 3.3 70B	Meta	67.7	86.4	89.5	89.9	90.7	+23.0
Scout 17B (16E)	Meta	65.4	77.4	80.5	82.9	83.3	+17.9
Maverick 17B (128E)	Meta	72.0	85.6	88.7	91.8	92.6	+20.6
Qwen3 32B	Alibaba	70.8	81.7	85.6	87.9	88.3	+17.5
Gemini 2.5 Flash	Google	63.8	86.8	91.8	93.4	93.8	+30.0
Gemini 2.5 Pro	Google	66.5	85.2	89.1	92.2	92.2	+25.7

B. MBPP Results

Table II presents results on MBPP Sanitized. The MBPP results reinforce and extend the findings from HumanEval:

Larger absolute gains. Self-repair gains on MBPP are substantially larger than on HumanEval for every model. Among open-weight models, gains range from +16.0 pp to +23.0 pp. The Gemini models show even larger gains: Flash achieves +30.0 pp and Pro +25.7 pp, the two highest in our study.

Flash dominates on MBPP. Gemini 2.5 Flash achieves the highest final pass rate (93.8%), despite starting at only 63.8%—lower than four of the five open-weight models. Its +30.0 pp repair gain exceeds the best open-weight model on MBPP (+23.0 pp) by 7 pp, suggesting qualitatively different repair behavior. Flash's R1 jump (+23.0 pp in a single round) accounts for the majority of its gain.

Consistent model ranking. The relative ordering of models is largely consistent across benchmarks. Llama 3.1 8B has the lowest final rate on both benchmarks, while Flash leads on both. On MBPP ($n = 257$), one problem corresponds to ∼0.4 pp, providing finer resolution for ranking comparisons.

C. Cross-Benchmark Comparison

Table III summarizes the self-repair gains across both benchmarks. MBPP consistently yields higher repair gains than HumanEval for every model. This difference likely reflects two factors: (1) lower initial pass rates on MBPP provide

TABLE III
CROSS-BENCHMARK COMPARISON OF SELF-REPAIR GAINS (Δ, IN PERCENTAGE POINTS) ON HUMANEVAL AND MBPP SANITIZED.

Model	Family	HumanEval Δ	MBPP Δ
Llama 3.1 8B	Meta	+9.8	+16.0
Llama 3.3 70B	Meta	+10.4	+23.0
Scout 17B (16E)	Meta	+14.0	+17.9
Maverick 17B (128E)	Meta	+6.7	+20.6
Qwen3 32B	Alibaba	+4.9	+17.5
Gemini 2.5 Flash	Google	+9.8	+30.0
Gemini 2.5 Pro	Google	+17.1	+25.7

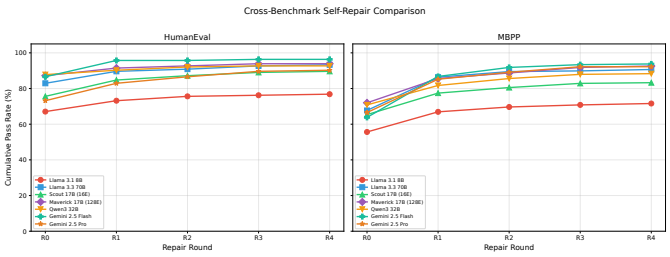

Fig. 2. Cross-benchmark comparison of self-repair gains for all seven models. MBPP consistently yields larger gains than HumanEval, with Gemini 2.5 Flash showing the largest MBPP gain (+30.0 pp).

more room for improvement, and (2) MBPP problems, while numerous, tend to be shorter and more focused, making their errors potentially easier to diagnose and repair. The Gemini models show the largest MBPP gains (+30.0 and +25.7 pp), suggesting that their error comprehension scales particularly well on shorter, focused problems.

Figure 2 visualizes the cross-benchmark comparison, highlighting the consistent pattern of larger MBPP gains across all models and families.

D. Error Distribution Analysis

Figure 3 shows the distribution of error types at the initial attempt (R_0) on HumanEval. Across all seven models, assertion errors dominate, indicating that most failures produce syntactically valid but logically incorrect code. Higher-capability models have fewer total failures but the same error-type profile, confirming that assertion errors are the irreducible core of failure regardless of model scale or family.

E. Repair Success by Error Type

Figure 4 analyzes repair success rates broken down by error type. A clear hierarchy emerges:

- **Name errors** are repaired at high rates ($\sim$77%). These errors provide specific diagnostic information (undefined variable or function names) that guides the model toward the fix.
- **Syntax errors** are repaired at moderate rates ($\sim$66%). While many syntax errors are straightforward to fix, some reflect deeper structural issues in code generation (e.g., incomplete functions or mismatched indentation) that are harder to resolve.

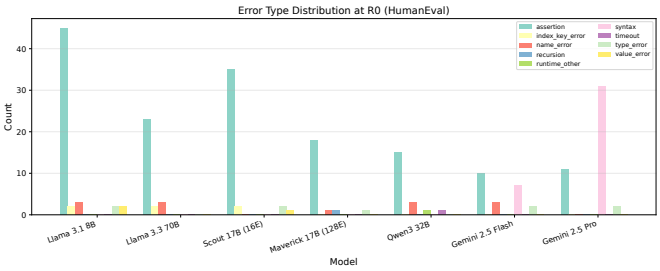

Fig. 3. Distribution of error types at the initial attempt (R_0) on HumanEval for each model.

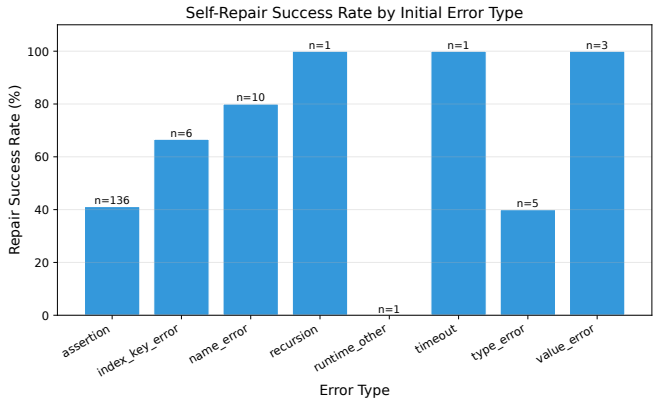

Fig. 4. Repair success rate by error type across all models. Assertion errors (logical mistakes) are the hardest to repair.

- **Assertion errors** are the hardest to repair, with success rates around 45%. An assertion error indicates that the code ran successfully but produced the wrong output. The error message provides minimal diagnostic information about *what* the code computed incorrectly or *why*, requiring the model to re-examine its logic.

This finding is consistent with Huang et al.'s observation [12] that LLMs struggle to self-correct reasoning without external feedback. The pattern holds across all seven models and both benchmarks, confirming that error type is a robust predictor of repair success regardless of model scale, family, or architecture.

F. Per-Round Improvement

Figure 5 shows the marginal improvement at each repair round on HumanEval. The pattern of diminishing returns is consistent across all models:

- $R_0 \rightarrow R_1$ **(first repair)**: The largest single-round gain for all seven models. Gemini 2.5 Pro gains +9.7 pp (the largest first-round gain), followed by Scout (+9.2 pp) and Flash (+9.1 pp), while Qwen3 gains +2.4 pp (the smallest, reflecting its high baseline).
- $R_1 \rightarrow R_2$ **(second repair)**: Still meaningful but reduced for all models.
- $R_2 \rightarrow R_3$ **and** $R_3 \rightarrow R_4$: Minimal additional gains. For Qwen3 and Maverick, R_4 yields no additional improve-

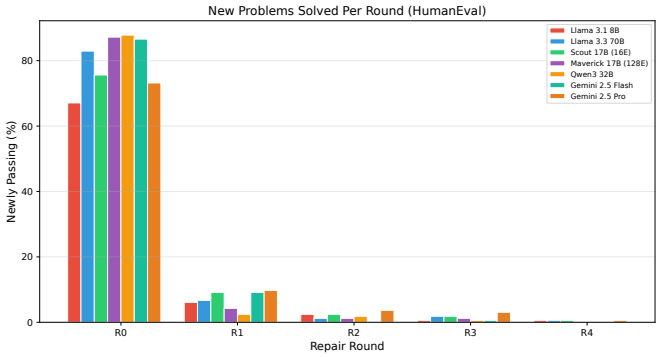

Fig. 5. Marginal improvement in pass@1 per repair round on HumanEval. The first repair round ($R_0 \rightarrow R_1$) yields the largest gain for all models.

TABLE IV
TOTAL TOKEN USAGE (PROMPT + COMPLETION) AND SELF-REPAIR GAIN ON HUMANEVAL (164 PROBLEMS). GEMINI MODELS' INTERNAL REASONING TOKENS ARE EXCLUDED FOR COMPARABILITY; SEE TEXT.

Model	Total Tokens	Δ (pp)	Tokens/pp
Llama 3.1 8B	195K	+9.8	19.9K
Llama 3.3 70B	112K	+10.4	10.8K
Scout 17B (16E)	155K	+14.0	11.1K
Maverick 17B (128E)	93K	+6.7	13.9K
Qwen3 32B	102K	+4.9	20.8K
Gemini 2.5 Flash	121K	+9.8	12.3K
Gemini 2.5 Pro	162K	+17.1	9.5K

ment over R_3, indicating that these models have solved all problems amenable to self-repair by round 3.

The concentration of gains in early rounds has practical implications: for most applications, two repair rounds capture the bulk of the benefit, and additional rounds offer diminishing marginal returns relative to their computational cost.

G. Token Cost Analysis

Table IV reports total token usage for each model on HumanEval. Token counts reflect prompt and completion tokens only. For Gemini models, internal reasoning ("thinking") tokens are excluded from this count; including them would increase Flash's total to ∼489K and Pro's to ∼994K. Since thinking tokens are not directly comparable to standard input/output tokens, we report prompt+completion for fair cross-model comparison. Gemini 2.5 Pro achieves the best cost-effectiveness at 9.5K tokens per percentage point of improvement, followed by Llama 3.3 70B at 10.8K tokens/pp.

Table V provides a fine-grained breakdown of when problems are first solved. The first repair round (R_1) accounts for the majority of all repairs across every model. Repair success rate—the fraction of initially-failed problems that are eventually fixed—correlates positively with model capability: Gemini 2.5 Flash repairs 72.7% of its failures (the highest), followed by Gemini 2.5 Pro at 63.6% and Llama 3.3 70B at 60.7%, while Llama 3.1 8B repairs only 29.6%.

TABLE V
PER-ROUND REPAIR BREAKDOWN ON HUMANEVAL: NUMBER OF PROBLEMS FIRST SOLVED AT EACH ROUND, AND THE REPAIR SUCCESS RATE (FRACTION OF INITIALLY-FAILED PROBLEMS EVENTUALLY REPAIRED).

Model	R_0	R_1	R_2	R_3	R_4	Never	Rep. %
Llama 3.1 8B	110	10	4	1	1	38	29.6
Llama 3.3 70B	136	11	2	3	1	11	60.7
Scout 17B (16E)	124	15	4	3	1	17	57.5
Maverick 17B (128E)	143	7	2	2	0	10	52.4
Qwen3 32B	144	4	3	1	0	12	40.0
Gemini 2.5 Flash	142	15	0	1	0	6	72.7
Gemini 2.5 Pro	120	16	6	5	1	16	63.6

TABLE VI
REPAIR PROMPT ABLATION ON HUMANEVAL. CUMULATIVE PASS@1 (%) AT EACH ROUND FOR UP TO THREE PROMPT STRATEGIES ON FOUR MODELS. SCOUT'S MINIMAL RUN IS UNAVAILABLE (SEE TEXT). DELTAS COMPUTED FROM RAW PROBLEM COUNTS BEFORE ROUNDING. R_0 AND SUBSEQUENT ROUNDS MAY VARY SLIGHTLY ACROSS RUNS DUE TO API-LEVEL NON-DETERMINISM; Δ COMPUTED FROM EACH RUN'S OWN R_0.

Model	Strategy	R_0	R_1	R_2	Δ
Llama 3.1 8B	Chain-of-thought	65.9	72.6	**77.4**	+11.6
	Explain-then-fix	65.9	72.0	76.2	+10.4
	Minimal	65.9	73.2	75.6	+9.8
Llama 3.3 70B	Chain-of-thought	84.1	93.9	**96.3**	**+12.2**
	Explain-then-fix	82.9	90.2	93.9	+11.0
	Minimal	81.1	86.6	87.8	+6.7
Scout 17B (16E)	Chain-of-thought	79.3	87.2	**89.6**	+10.4
	Explain-then-fix	78.0	87.2	87.2	+9.1
Qwen3 32B	Chain-of-thought	89.6	97.0	**97.0**	+7.3
	Explain-then-fix	88.4	93.3	94.5	+6.1
	Minimal	87.8	90.2	91.5	+3.7

H. Repair Prompt Ablation

Our main experiments use a minimal repair prompt that provides only the error message and asks for a corrected function. To assess sensitivity to prompt design, we compare up to three repair prompt strategies on four models—Llama 3.1 8B (dense, 8B), Llama 3.3 70B (dense, 70B), Llama 4 Scout (MoE, 17B active), and Qwen3 32B (dense, 32B)—across all 164 HumanEval problems with up to two repair rounds (R_0–R_2). All three strategies were evaluated for three models; for Scout, only CoT and Explain-then-fix are available due to an incomplete experimental run.

1) **Minimal**: The baseline strategy used throughout this paper. The repair prompt presents the error message and requests a corrected function.
2) **Explain-then-fix**: The model is asked to first explain the bug in 1–2 sentences, then provide the corrected code. This follows the explain-before-fix paradigm from Self-Debug [6].
3) **Chain-of-thought (CoT)**: The model is prompted to reason step-by-step: (1) what does the error tell us, (2) what is the root cause, (3) what is the fix. The corrected code follows the analysis.

Table VI and Figure 6 present the results across four models.

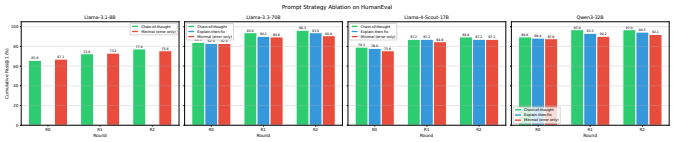

Fig. 6. Prompt strategy ablation on HumanEval. CoT consistently achieves the highest final pass rate across all models, with the benefit scaling with model capability.

Note that each ablation configuration involves a complete re-execution of all 164 problems, including initial generation at R_0. Since the repair strategy does not affect the initial generation prompt, any R_0 variation across strategies for the same model reflects non-determinism in the Groq API's served model weights across sessions. For example, Llama 3.3 70B shows R_0 values of 84.1%, 82.9%, and 81.1% across the three strategies. The Δ column is computed relative to each run's own R_0, making repair gain measurements independent of this baseline variation. Crucially, the key finding—that CoT achieves the highest final R_2 rate for every model—holds regardless of which R_0 baseline is used.

CoT consistently wins across all models. Chain-of-thought achieves the highest final pass rate for every model: 77.4% (8B), 96.3% (70B), 89.6% (Scout), and 97.0% (Qwen3). Qwen3 with CoT achieves the highest pass rate of any model-strategy combination in our ablation (97.0% at R_2).

The benefit of richer prompts scales with model capability. Measured by the additional self-repair gain ($\Delta_{\text{CoT}} - \Delta_{\text{minimal}}$), the advantage of CoT is modest for the 8B model (+1.8 pp) but widens for stronger models: +5.5 pp for Llama 3.3 70B and +3.6 pp for Qwen3. In absolute terms, CoT achieves final R_2 rates 1.8–8.5 pp higher than minimal, though part of this gap reflects R_0 variation across runs (see above). This confirms the interaction between model capability and prompt strategy across the three models with complete ablation data, spanning 8B to 70B parameters.

The practical implication is clear: for capable models ($\geq$32B), chain-of-thought repair prompting is strongly recommended and can yield gains comparable to an additional 1–2 repair rounds. For smaller models, the benefit is present but modest, and the simpler minimal prompt remains a competitive choice.

I. Self-Repair vs. Independent Resampling

A key question is whether the token budget spent on iterative repair could be better used by simply drawing multiple independent samples. To address this, we compare self-repair (greedy decoding, $T = 0$, up to 5 sequential attempts with error feedback) against independent resampling ($T = 0.8$, 5 independent samples) on HumanEval. This comparison involves two differences: error feedback (present in repair, absent in resampling) and decoding temperature ($T = 0$ vs. $T = 0.8$). A fully controlled study would also require stochastic repair ($T > 0$ with feedback); we leave this to future work. Nevertheless, this comparison addresses the practical question: given a token budget, should one invest in sequential repair or parallel resampling? For resampling, we report the unbiased pass@k estimator [1].

TABLE VII
SELF-REPAIR (GREEDY, 5 ROUNDS) VS. INDEPENDENT RESAMPLING (TEMPERATURE 0.8, 5 SAMPLES) ON HUMANEVAL (164 PROBLEMS). BOLD INDICATES HIGHER PASS RATE. NOTE: REPAIR USES GREEDY DECODING ($T = 0$) WHILE RESAMPLING USES $T = 0.8$; SEE TEXT.

Model	Self-Repair		Resampling	
	Final %	Tokens	pass@5 %	Tokens
Llama 3.1 8B	76.8	195K	**79.9**	219K
Llama 3.3 70B	**93.3**	112K	90.9	231K
Scout 17B (16E)	**89.6**	155K	86.0	255K
Qwen3 32B	92.7	102K	92.7	223K

Table VII and Figure 7 compare self-repair against independent resampling on HumanEval for four models.[2] The results reveal a clear, model-dependent pattern:

Self-repair is always more token-efficient. Across all four models, self-repair uses fewer tokens than resampling to achieve comparable or better pass rates, with savings ranging from 11% (8B) to 54% (Qwen3). The savings are largest for models whose strong error comprehension enables efficient single-attempt repairs, while the 8B model's longer repair traces narrow the gap. This efficiency advantage arises because self-repair generates only the corrected function at each round, while resampling generates a complete solution from scratch each time.

Self-repair becomes increasingly competitive with model capability. For the weakest model (Llama 3.1 8B), resampling achieves a modestly higher pass@5 (79.9% vs. 76.8%)—the diversity of independent samples compensates for the model's limited error comprehension. For the three stronger models, self-repair matches or exceeds resampling: Scout (89.6% vs. 86.0%), Llama 3.3 70B (93.3% vs. 90.9%, a difference of 4 problems), and Qwen3 32B (92.7% tied) all favor repair. The 70B result is particularly striking: self-repair achieves +2.4 pp higher pass rate while using less than half the tokens (112K vs. 231K).

This pattern has a clear interpretation: as models become more capable, their ability to diagnose and fix errors from feedback improves, making the *informational signal* from error messages more valuable than the *diversity* from independent samples. We note that this comparison does not fully disentangle the value of error feedback from the effect of decoding temperature; a stochastic repair condition would be needed to isolate these factors. The practical implication is that self-repair is the preferred strategy for capable models, while a hybrid approach—repairing diagnosable errors and resampling for assertion errors—may be optimal for weaker models.

[2]Maverick was unavailable on the Groq free-tier API during the resampling experiments. The four evaluated models span the full capability range (8B–70B) and both architecture types (dense and MoE), providing sufficient evidence for the observed patterns.

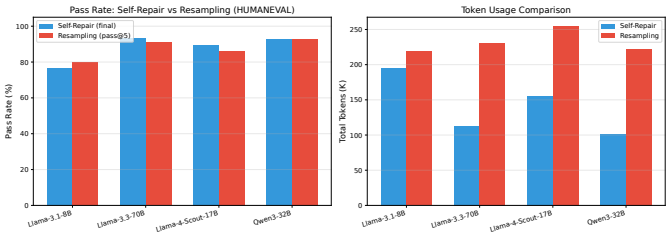

Fig. 7. Self-repair vs. independent resampling on HumanEval. Self-repair matches or exceeds resampling for all models except the 8B, while consistently using fewer tokens.

VI. Discussion

A. Self-Repair as a Universal Improvement Strategy

Our results demonstrate that iterative self-repair with error feedback is a universally effective strategy for improving LLM code generation across all seven models tested, spanning three families (Meta, Alibaba, Google), both open-weight and proprietary models, and both dense and MoE architectures. This finding updates the conclusions of Olausson et al. [5], who found that weaker models could be harmed by self-repair. The discrepancy likely reflects improvements in instruction-following capabilities of recent model generations: all seven models in our study demonstrate competent error comprehension, even at the 8B scale. All models in Olausson et al.'s study were released in 2022–2023; our evaluation covers 2024–2025 models, a period that saw significant improvements in instruction-following and error comprehension capabilities.

The cross-benchmark consistency of our findings strengthens this conclusion. Self-repair improves every model on both HumanEval and MBPP, with largely consistent model rankings across benchmarks. The inclusion of Gemini models from a third family further strengthens generalizability: self-repair effectiveness is a robust property of modern LLMs rather than an artifact of a particular model family or benchmark.

B. Dense vs. Mixture-of-Experts Architectures

Our evaluation includes dense models (Llama 8B, 70B, Qwen3 32B) and MoE models (Scout 16E, Maverick 128E) among the open-weight models, as well as two proprietary Gemini models whose architectures have not been publicly disclosed, enabling comparisons across three families. Several observations are noteworthy:

- Among open-weight models, Scout (16 experts) achieves the highest repair gain on HumanEval (+14.0 pp), suggesting that its MoE architecture particularly benefits from the opportunity to repair its errors.
- Gemini 2.5 Flash achieves the highest final pass rates on both benchmarks (96.3% HumanEval, 93.8% MBPP), surpassing all open-weight models. Its combination of high initial accuracy and strong repair capability sets a new ceiling.
- Gemini 2.5 Pro, despite being a frontier reasoning model, underperforms Flash on both benchmarks (90.2% vs. 96.3% on HumanEval). This suggests that reasoning overhead can hurt in a simple prompt-completion paradigm, echoing our findings with Qwen3's thinking mode.
- The 70B dense model achieves comparable final performance to Maverick on both benchmarks, suggesting that for code generation, a large dense model can match a smaller MoE model with many experts.

These comparisons are suggestive but must be interpreted cautiously: models differ not only in architecture but also in training data, parameter counts, and other confounds that we cannot isolate. We report these observations as correlations rather than causal claims.

TABLE VIII
LiveCodeBench results (50 problems). Self-repair improves all four models evaluated.

Model	R_0	Final	Δ
Llama 3.1 8B	10.0	12.0	+2.0
Scout 17B (16E)	6.0	14.0	+8.0
Qwen3 32B	8.0	10.0	+2.0
Gemini 2.5 Flash	0.0	16.0	+16.0

C. Qualitative Analysis of Repair Outcomes

To complement our quantitative findings, we examine representative cases from HumanEval.

Successful repairs. On HumanEval/38, Llama 3.3 70B produced code that failed at R_0 with a `NameError` and repaired it in a single round. More interestingly, on HumanEval/26, Llama 3.1 8B failed with assertion errors for three consecutive rounds before finally producing a correct solution at R_3, demonstrating that persistence can resolve even logical errors. On HumanEval/77, Scout exhibited error-type mutation across rounds (type error $\rightarrow$ assertion $\rightarrow$ type error $\rightarrow$ pass), showing the model genuinely restructuring its approach rather than making superficial edits.

Persistent failures. On HumanEval/10, Llama 3.1 8B produced assertion errors on all five attempts, never resolving the underlying logic error. On HumanEval/32, Maverick exhibited error *thrashing*: assertion $\rightarrow$ name error $\rightarrow$ assertion $\rightarrow$ assertion $\rightarrow$ timeout—the failure mode described by Olausson et al. [5]. We identified several "universally hard" problems (e.g., HumanEval/132, HumanEval/145) that defeated all models even with four repair rounds.

D. Preliminary Evaluation on Harder Benchmarks

To assess whether self-repair generalizes beyond the relatively easy HumanEval and MBPP benchmarks, we conducted a preliminary evaluation on LiveCodeBench [22], a contamination-free benchmark of competitive programming problems. We evaluated four models on 50 LiveCodeBench problems (18 easy, 23 medium, 9 hard) with the same 5-round self-repair protocol. We emphasize that this evaluation is exploratory: the small sample size (50 problems) limits statistical power.

Table VIII shows that self-repair improves all four models on LiveCodeBench. Gemini 2.5 Flash achieves the largest gain

(+16.0 pp), reaching 16% from an initial pass rate of 0%—all solved problems come entirely from repair rounds. Flash's 0% R_0 reflects the difficulty of competitive programming problems combined with stdin/stdout format requirements, yet its repair capability recovers 8 problems. Scout achieves the second-largest gain (+8.0 pp), more than doubling its pass rate from 6% to 14%.

While the absolute pass rates are much lower than on HumanEval (as expected for competitive programming), self-repair remains effective. The relative repair gains are actually larger than on HumanEval, suggesting that self-repair *may* scale to harder benchmarks where the lower baseline leaves more room for improvement. At $n = 50$, each problem corresponds to 2 pp, so differences smaller than $\sim$4 pp should be interpreted cautiously.

E. Practical Implications

Our findings suggest several practical guidelines for deploying LLM code generation systems:

1) **Always include at least one repair round**: The first repair round consistently provides the largest marginal improvement for all models tested.
2) **Two rounds capture most gains**: For cost-sensitive applications, two repair rounds (R_0 through R_2) capture the majority (76–94%) of the total achievable improvement.
3) **Consider error type for routing**: Name errors are repaired at the highest rates ($\sim$77%), while assertion errors are the hardest ($\sim$45%). For assertion-dominated failures, alternative strategies (e.g., sampling multiple independent solutions) may be more cost-effective.
4) **Model selection matters**: For applications where final pass rate is paramount, Gemini 2.5 Flash (96.3%) or Maverick (93.9%) should be preferred. For self-repair gain per token, Gemini 2.5 Pro and Llama 3.3 70B offer the best efficiency.

VII. Limitations and Future Work

We identify several limitations of this study that suggest directions for future work.

Resampling comparison scope. Our resampling comparison (Section V, Table VII) covers four of seven models on HumanEval only. While the results clearly show that self-repair is more token-efficient and increasingly advantageous for stronger models, extending this comparison to MBPP and controlling for token budget more precisely (e.g., allocating the exact same number of tokens to each strategy) would strengthen the analysis. Additionally, we compare greedy repair against stochastic resampling (temperature $= 0.8$), which conflates two differences—error feedback vs. diversity, and greedy vs. stochastic decoding. A fairer comparison might use stochastic repair as well.

Benchmark scope. Our main evaluation uses HumanEval and MBPP, which are considered relatively easy by 2025 standards. Our preliminary LiveCodeBench evaluation (Section VI) shows that self-repair remains effective on harder competitive programming problems, but covers only four models on 50 problems. A comprehensive evaluation across all models on LiveCodeBench [22], BigCodeBench [23], or SWE-Bench [24] would more fully characterize self-repair on problems requiring deep algorithmic reasoning.

Greedy decoding only. Our use of temperature $= 0.0$ ensures deterministic, reproducible results but eliminates sampling variance, precluding confidence intervals or statistical significance tests. On HumanEval (164 problems), a single problem corresponds to $\pm$0.6 pp, meaning differences of $\sim$1.2 pp or less (e.g., Maverick at 93.9% vs. Qwen3 at 92.7%, a gap of just two problems) should not be over-interpreted. Experiments with stochastic decoding across multiple seeds would provide more robust estimates and enable statistical analysis.

Ablation scope. Our prompt ablation covers four of seven models (8B, 70B, Scout, Qwen3) with two repair rounds (R_0–R_2). Extending to all five repair rounds would characterize whether CoT's advantage persists or converges in later rounds. The ablation uses HumanEval only; prompt sensitivity on MBPP or LiveCodeBench may differ.

API-served model weights. Open-weight models were accessed via the Groq free-tier API, which may serve quantized or otherwise optimized model variants. We did not verify that the served weights exactly match the canonical model releases. Gemini models were accessed via Google Cloud Vertex AI, where the served model versions are controlled by Google. Pass rates may differ slightly across API versions or on locally-hosted weights.

Model family diversity. Our evaluation covers three model families (Llama, Qwen, and Gemini) across two inference providers (Groq and Vertex AI), including both open-weight and proprietary models. Code-specialized models (e.g., DeepSeek-Coder [25]) may exhibit different repair dynamics due to code-focused pre-training and remain a direction for future work.

Qwen3 reasoning mode. We evaluated Qwen3 with thinking mode enabled on all 164 HumanEval problems. Thinking mode achieves a *lower* final pass rate (89.0% vs. 92.7% for no-think), despite a much larger repair gain (+14.0 pp vs. +4.9 pp). The explanation lies in the initial pass rate: thinking mode's R_0 drops to 75.0% (vs. 87.8% for no-think) because long reasoning traces contaminate the code output, causing syntax errors. Self-repair compensates by fixing 23 additional problems, but 18 problems fail due to API errors or persistent extraction issues. Our decision to disable thinking mode in the main experiments is validated: with the current code extraction pipeline, thinking mode reduces net performance. A more robust extraction approach (or native tool-use integration) could potentially unlock the reasoning benefits without the extraction overhead.

VIII. Threats to Validity

Internal validity. Our use of greedy decoding ensures deterministic, reproducible results but reflects a single point in the sampling distribution. Prior work [5], [6] similarly reports single-run results under fixed decoding settings. Our prompt ablation (Table VI) shows that repair prompt design does affect

outcomes, meaning the main results represent a lower bound for capable models. Additionally, minor R_0 variation in the ablation table suggests the Groq API may serve slightly different model versions across sessions.

External validity. Our model selection (seven models from three families) spans a meaningful range, including both open-weight and proprietary models, but may not capture behaviors of all architectures. We do not evaluate code-specialized models (e.g., DeepSeek-Coder) or models with native tool use. Additionally, because HumanEval and MBPP are widely known, evaluated models may have been exposed to these problems during pre-training, which could inflate initial pass rates. However, this affects all models equally and does not invalidate relative comparisons.

Construct validity. Our cumulative pass@1 metric counts a problem as solved if it passes all provided test cases. However, test suites may not cover all edge cases, meaning some "passing" solutions could contain latent bugs. Our error classification relies on Python exception types, which may conflate distinct failure modes.

IX. Conclusion

We have presented a systematic study of iterative self-repair across seven models from three families (Meta, Alibaba, Google), spanning both open-weight and proprietary models, dense and MoE architectures, and two benchmarks (HumanEval and MBPP Sanitized). Self-repair is universally effective: every model improves on every benchmark. Error-type analysis confirms that name errors are repaired at high rates, while assertion errors remain the most challenging—connecting to broader findings on the limits of LLM self-correction.

Two repair rounds capture the majority of achievable gains, providing clear deployment guidance. Our prompt ablation shows that chain-of-thought repair can yield additional gains for capable models, though the benefit is model-dependent. These results establish iterative self-repair as a practical, training-free technique for improving LLM code generation, while identifying code-specialized models and harder-benchmark evaluation as important directions for future work.

Acknowledgment

The author thanks Groq for providing free-tier API access that enabled the open-weight model experiments, and Google Cloud for Vertex AI credits used for the Gemini experiments. All experimental code and results are publicly available at https://github.com/Johin2/Self-repair. The author declares no conflicts of interest.

References

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with AlphaCode," in *Science*, vol. 378, no. 6624, 2022, pp. 1092–1097.

[3] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2024.

[4] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[5] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" *arXiv preprint arXiv:2306.09896*, 2024.

[6] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," *arXiv preprint arXiv:2304.05128*, 2024.

[7] A. Grattafiori, A. Dubey, A. Jauhri *et al.*, "Llama 3 model card," *arXiv preprint arXiv:2407.21783*, 2024.

[8] Meta AI, "Llama 3.3: Large language model," *Meta AI Blog*, 2024, https://ai.meta.com/blog/llama-3-3/.

[9] ——, "Llama 4: Maverick, scout, and behemoth," *Meta AI Blog*, 2025, https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

[10] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[11] Google DeepMind, "Gemini 2.5: Our most intelligent AI model," *Google Blog*, 2025, https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/.

[12] J. Huang, C. Xia, P. Shenoy, and J. Zhao, "Large language models cannot self-correct reasoning yet," *arXiv preprint arXiv:2310.01798*, 2024.

[13] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 21 314–21 328, 2022.

[14] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Zou, D. Song, and J. Steinhardt, "Measuring coding challenge competence with APPS," *arXiv preprint arXiv:2105.09938*, 2021.

[15] S. Kulal, P. Pasupat, K. Chandra, M. Lee, O. Padon, A. Aiken, and P. Liang, "SPoC: Search-based pseudocode to code," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[16] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[17] N. Shinn, F. Cassano, A. Gopinath, K. Shakkottai, A. Labash, and S. Liu, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[18] D. Key, S. Lee, J. Shin, and S. Hwang, "Multi-turn code generation with single-turn instruction tuning," *arXiv preprint arXiv:2404.11137*, 2024.

[19] Y. Zhang, J. Chen, D. Li, and Y. Zheng, "Self-collaboration code generation via ChatGPT," *arXiv preprint arXiv:2304.07590*, 2023.

[20] L. Zhong, Z. Wang, and J. Shang, "LDB: A large language model debugger via verifying runtime execution step-by-step," *arXiv preprint arXiv:2402.16906*, 2024.

[21] S. Jiang, Y. Wang, and Y. Wang, "Selfevolve: A code evolution framework via large language models," *arXiv preprint arXiv:2306.02907*, 2023.

[22] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and contamination free evaluation of large language models for code," *arXiv preprint arXiv:2403.07974*, 2024.

[23] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," *arXiv preprint arXiv:2406.15877*, 2024.

[24] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" *arXiv preprint arXiv:2310.06770*, 2024.

[25] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.